

CMPE476 C10K Benchmark Report

1. Test Setup

- Machine: ASUS - System Product Name
- CPU / RAM / Exchange: 12 / 16211MB / 4096MB
- OS and kernel: Ubuntu on WSL2 (Linux Duran-Desktop 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux)
- `ulimit -n`: 10240
- Any `sysctl` tuning: none

2. Methodology

- Built with `make` from the submitted source.
- Load generator: `client_flood`.
- For each server (`threadserv`, `epollserv`) and each concurrency level (1000, 5000, 10000):
 - launch server
 - run `client_flood <host> <port> <clients> <requests_each>`
 - record requests/sec and RSS from `/proc/<pid>/status`
 - stop server
- Requests per client: 100.
- Host: 127.0.0.1.

3. Results Table

Server	Clients	Requests Each	Completed		RPS	RSS	RSS
			Re- quests	Elapsed (s)		Before (kB)	After (kB)
threadserv	1000	100	100000	0.330	303030	1408	3648
threadserv	5000	100	500000	1.332	375375	1408	7184
threadserv	10000	100	1000000	2.670	374532	1408	9236
epollserv	1000	100	100000	0.198	505051	1408	3748
epollserv	5000	100	500000	0.965	518135	1408	16768
epollserv	10000	100	1000000	1.918	521376	1408	32256

4. Chart

Throughput vs Clients (RPS)

Clients	Threadserv RPS	Epollserv RPS
1000	303030	505051
5000	375375	518135

Clients	Threadserv RPS	Epollserv RPS
10000	374532	521376

RSS After Run vs Clients (kB)

Clients	Threadserv RSS (kB)	Epollserv RSS (kB)
1000	3648	3748
5000	7184	16768
10000	9236	32256

5. Analysis

The epoll-based server outperformed the thread-per-connection server at all tested concurrency levels.

- At 1000 clients, **epollserv** reached 505,051 RPS vs 303,030 RPS (~66.7% higher).
- At 5000 clients, **epollserv** reached 518,135 RPS vs 375,375 RPS (~38.0% higher).
- At 10000 clients, **epollserv** reached 521,376 RPS vs 374,532 RPS (~39.2% higher).

Thread-per-connection performance improves from 1k to 5k clients, then plateaus around ~375k RPS at 10k clients, which is consistent with scheduler overhead and thread-management costs dominating under high concurrency. In contrast, the epoll event loop remains scalable because one thread handles readiness events for many sockets without per-connection thread scheduling overhead.

RSS-after-run values in this measurement are higher for **epollserv** at larger client counts. This is opposite to the common expectation and is likely influenced by measurement methodology:

- This report captures RSS near/after run completion, not peak RSS during active load.
- Allocator behavior can keep arenas/pages mapped even after connections close.
- The event-loop implementation may allocate per-connection buffers that increase resident memory at high fan-out.

Therefore, throughput conclusions are strong, while memory conclusions should be interpreted carefully unless peak RSS is sampled continuously during each run.